

Fast and accurate implementation of the Dirichlet multinomial log-likelihood function

Sherenaz Al-Haj Baddar

Department of Computer Science, The University of Jordan, Amman, 11942, Jordan

Alessandro Languasco*

Dipartimento di Matematica, Università di Padova, via Trieste 63, Padova, 35121, Italy

Mauro Migliardi

Dipartimento di Ingegneria dell'Informazione, Università di Padova, via Gradenigo 6/b, Padova, 35131, Italy

Abstract

Fitting overdispersed count data accurately and efficiently is instrumental for statistical modeling. Failing to capture the overdispersed nature of data yields to inaccurate model parameter and error estimations, and this jeopardizes the validity and reliability of statistical inferences. In this study, we expand on previous studies on developing efficient and accurate estimations of the Dirichlet Multinomial distribution *log-likelihood* function. A faster and accurate implementation of a state-of-the-art technique is presented and its performance is compared to Python's math and Scipy implementations of the same function. Experiments conducted using 10 different public datasets, two of which are not overdispersed, show that the proposed technique achieves a 1.5 up to 10 fold speed gain over both of Python's implementations of the *log-likelihood* function.

Keywords: Dirichlet multinomial distribution, overdispersion, log-likelihood function, fixed-point iteration, Python-wrapping of C code.

1. Introduction

We continue our journey into improving the current state of knowledge about how to compute the *Dirichlet-multinomial log-likelihood* function. In [8] we showed the Languasco-Migliardi (LM) general algorithm and in [1] we compared the Python implementations of our technique and the one by Yu-Shaw [18] (YS). In [2] we focused on the problem of modeling and forecasting IoT-data. The experiments contained in [1, 2]

*Corresponding author

Email addresses: s.baddar@ju.edu.jo (Sherenaz Al-Haj Baddar),
alessandro.languasco@unipd.it (Alessandro Languasco), mauro.migliardi@unipd.it (Mauro Migliardi)

showed the superiority of the **LM** technique over **YS** on several sets of overdispersed data, thus, confirming the theoretical results contained in [8].

Denoting as Γ the Euler Gamma function and as $\log$ the natural logarithm, we adopted Minka’s [11] procedure (MP, see Algorithm 1 below) and inserted the two Python implementations of the **LM** technique for computing $\log \Gamma$ into the Dirichlet-multinomial log-likelihood function. Considering that there exists a publicly available implementation of MP, see [14]¹, that uses the `python-scipy` library code of the $\log \Gamma$ function (`gammaLn`), we thought to compare it to the **LM** approach. To achieve this goal, we implemented the latter using the C language and wrapped it in Python. Then, embodying it into MP, we compared the accuracy and the efficiency of such a Python-wrapped C-implementation of the **LM** technique with the ones of the code provided in [14].

Our experiments showed that the `python-math` library implementation of $\log \Gamma$ (`lgamma`) is at least twice faster than `gammaLn`. We were quite surprised by this also because, according to their documentations, both functions refer to the same C implementation of $\log \Gamma$. However, it seems to us that `python-math` refers to Lanczos’ algorithm [7] for Γ , see [15] – they compute $\Gamma(u)$ first (a fact that might create further overflow problems, though), and then they evaluate the log-function on the result – while Scipy seems to always use the Stirling approximation of $\log \Gamma(u)$, see [16]. Summarising, it seems that `lgamma` and `gammaLn` were obtained by implementing two different algorithms; we suspect that this is the reason why we observed a major difference in their performances. Considering this situation, to perform fairer and meaningful experiments we decided to use both implementations in the MP procedure² and to compare their results with the ones obtained by using our Python-C wrapped implementation of the **LM** technique as presented in [8, 1]. As in [1, 2], in this case too, all these codes were tested using several public datasets coming from different sources and pertaining to different application fields.

For the other special function needed in Algorithm 1, i.e. the *digamma* function F^3 , and as in [14], the implementation provided by the `python-scipy` package (`digamma`) was used. In this way, working into the MP-framework, only the $\log \Gamma$ -computation aspect of highly optimised and widely distributed Python-C-portings and the one obtained using the **LM** technique were compared.

In the following, the Python Scipy version of Minka’s code in [14] is denoted by as **MP-scipy**, while its variant obtained by inserting `lgamma` is denoted by **MP-math**. Moreover, the variant that uses the **LM** technique is denoted by **MP-ALM**.

The Experiments conducted in this study show that **MP-ALM** is always faster than **MP-scipy** with a performance gain that varies from a factor of 2.21 to 11.60. Furthermore, in general, larger gains are attained with overdispersed datasets, while smaller

¹A `python-scipy` version of the original Minka’s Fastfit Matlab toolbox, see <https://tminka.github.io/software/fastfit/>.

²We just modified the code in [14] by importing the `python-math` library and replacing each call to `gammaLn` with one to `lgamma`.

³In the literature the digamma function $F(u)$ is also denoted as $\psi(u)$ but in this context ψ has a different meaning.

gains are associated with non-overdispersed datasets.⁴ Comparing with **MP-math**, the gain **MP-ALM** achieved ranged from a factor of 1.04 to 5.02. Again, the larger gains were observed with overdispersed datasets, whereas the smaller ones were associated with the non-overdispersed datasets.

This paper is organized as follows: Section 2 describes the Minka procedure and its fixed-point Iteration algorithm; it also revises the mathematical background behind the **LM** technique. Section 3 depicts a Python C-wrapped implementation of the **DM** distribution *log-likelihood* function using the **LM** technique, while shedding light on practical details pertaining to the speed and accuracy of this implementation. Section 4 describes the datasets used in this study, illustrates the experiments conducted, and discusses their results. A summary on recent relevant studies is presented in Section 5, while Section 6 depicts concluding remarks and highlights ideas for future work.

2. Minka’s procedure to evaluate the Dirichlet Multinomial Distribution Parameters

We need first to give some formal definition. In the following we will denote $\mathbb{N}$ as the set of natural numbers and $\mathbb{N}^*$ as $\mathbb{N} \setminus \{0\}$. Let $K \in \mathbb{N}^*$, $\mathbf{x} := (x_1, \dots, x_K)$, $x_k \in \mathbb{N}^*$ for $k = 1, \dots, K$, $\boldsymbol{\alpha} := (\alpha_1, \dots, \alpha_K)$, $\alpha_k > 0$ for $k = 1, \dots, K$, $N := \sum_{k=1}^K x_k$ and $A := \sum_{k=1}^K \alpha_k$. Letting $\psi := 1/A$ be the *overdispersion* parameter and $\mathbf{p} := (p_1, \dots, p_K)$, $p_k > 0$ for $k = 1, \dots, K$, such that $\sum_{k=1}^K p_j = 1$, we also set $\alpha_k = p_k/\psi$, $k = 1, \dots, K$. We denote the *Dirichlet-multinomial log-likelihood* function as

$$\begin{aligned} \log \mathcal{L}(\mathbf{p}, \psi, \mathbf{x}) := & -\left(\log \Gamma\left(\frac{1}{\psi} + N\right) - \log \Gamma\left(\frac{1}{\psi}\right)\right) \\ & + \sum_{k=1}^K \left(\log \Gamma(\alpha_k + x_k) - \log \Gamma(\alpha_k)\right). \end{aligned}$$

Modeling a dataset using a statistical distribution comprises estimating the parameters that maximize its likelihood. The work in [11] presents an algorithm to estimate the Dirichlet multinomial distribution parameters in order to model overdispersed categorical count data. In order to estimate the Dirichlet distribution parameter $\boldsymbol{\alpha}$, a subset of the input is selected at random. Next, an iterative algorithm is applied to this subset in order to estimate $\boldsymbol{\alpha}$ to a preset degree of accuracy [11]. In this work, the fixed-point algorithm described in [11] and illustrated in Algorithm 1 is employed.

In addition to a randomly chosen subset of the original dataset with R rows and K columns, denoted by **data**, the fixed-point algorithm requires an initial guess for the $\boldsymbol{\alpha}$ parameter. It also utilizes a maximum number of iterations, denoted by $\mathcal{T}$, and an upper limit (i.e., a threshold) on the acceptable difference between two consecutive estimates of $\boldsymbol{\alpha}$, denoted by Δ .

⁴A case for which the **LM** approach was not specifically designed for!

Algorithm 1 A Fixed-point Iteration Algorithm for estimating the **DM** parameter α

ESTIMATE- α (**data**, $\mathbf{x}$, $\mathcal{T}$, R , K , Δ , α_0)

- **data**: a subset of the original dataset; a matrix of counts with R rows and K columns
- $\mathbf{x}$: a vector of length K , where x_k is the summation of all counts in column k in **data**
- $\mathcal{T}$: maximum number of iterations until convergence
- Δ : an upper limit on the difference between two consecutive estimates of α
- α_0 : initial guess for α
- returns (α, ψ) , the **DM** distribution parameters if ESTIMATE- α converges, otherwise, returns α_{error}

```

1: for  $j = 1, \dots, \mathcal{T}$ 
2:    $\Sigma_1 \leftarrow 0; \Sigma_2 \leftarrow 0$ 
3:    $\Sigma_{\alpha_0} \leftarrow \sum_{k=1}^K \alpha_0[k]$ 
4:   for  $k = 1, \dots, K$ 
5:     for  $i = 1, \dots, R$ 
6:        $\Sigma_i \leftarrow \sum_{k=1}^K \mathbf{data}[i, k]$ 
7:        $\Sigma_1 \leftarrow \Sigma_1 + F(\mathbf{data}[i, k] + \alpha_0[k]) - F(\alpha_0[k])$ 
8:        $\Sigma_2 \leftarrow \Sigma_2 + F(\Sigma_i + \Sigma_{\alpha_0}) - F(\Sigma_{\alpha_0})$ 
9:     endfor  $i$ 
10:     $\alpha_1 \leftarrow \alpha_0 \cdot (\Sigma_1 / \Sigma_2)$ 
11:  endfor  $k$ 
12:   $\psi_0 \leftarrow 1 / \sum_{k=1}^K \alpha_0[k]; \mathbf{p}_0 \leftarrow \psi_0 \cdot \alpha_0$ 
13:   $\psi_1 \leftarrow 1 / \sum_{k=1}^K \alpha_1[k]; \mathbf{p}_1 \leftarrow \psi_1 \cdot \alpha_1$ 
14:  if  $(|\log \mathcal{L}(\mathbf{p}_0, \psi_0, \mathbf{x}) - \log \mathcal{L}(\mathbf{p}_1, \psi_1, \mathbf{x})| < \Delta)$ 
15:    return  $[\alpha_1, \psi_1]$ 
16:  else
17:     $\alpha_0 \leftarrow \alpha_1$ 
18:  endfor  $j$ 
19: return  $\alpha_{\text{error}}$ 

```

As depicted in Algorithm 1, no more than $\mathcal{T}$ iterations are executed, in each iteration a new estimate for the parameter α , denoted by α_1 , is generated. This is achieved by scanning **data** in order to calculate the quantities Σ_1 and Σ_2 which accumulate two paired differences of the digamma function F , as described in Eq. (55) in [11]. These two quantities are then used to estimate a new α (i.e., α_1). Next, the absolute paired difference of the log-likelihood of the **DM** distribution at both α_0 and α_1 is calculated and compared to the threshold Δ ; if that difference is less than Δ , then the algorithm is assumed to have converged. In this case, it returns α_1 and the accompanying parameter

ψ_1 . However, if no such convergence is reached by $\mathcal{T}$ iterations, the algorithm is assumed to have failed and the value α_{error} is returned.

The soundness of the fixed-point iteration is jeopardized when the data is overdispersed since the paired difference, depicted in line 14, comprises not one, but two calls to the **DM** $\log \mathcal{L}$ function. Each such call is prone to the lack-of-accuracy problem detailed in [8].

Clearly, we cannot here repeat the whole algorithm presented in [8], but, for completeness, we write here the main result proved there. For $x \geq 0$, $y \in \mathbb{N}^*$, $m \in \mathbb{N}^*$, we define

$$d(x, y) := (1 + x(y - 1))^{-1}, \quad (1)$$

$$S_{2m}(x, y) := \sum_{n=1}^m \frac{B_{2n} x^{2n-1}}{2n(2n-1)} (d(x, y)^{2n-1} - 1), \quad (2)$$

where B_{2n} are the even-index *Bernoulli numbers*; we recall that, with the unique exception of $B_1 = -1/2$, the odd-index Bernoulli numbers are equal to 0.

Moreover, for $x > 0$, we define

$$E_{2m}(x, y) := \frac{1}{2m} \int_{1/x}^{1/x+y-1} B_{2m} \left(\left\{ t - \frac{1}{x} \right\} \right) \frac{dt}{t^{2m}}, \quad (3)$$

where we denoted as $\{u\}$ the fractional part of $u \in \mathbb{R}$ and $B_{2n}(u)$ are the even-index *Bernoulli polynomials*. We remark that $S_{2m}(x, 1) = 0$ and $E_{2m}(x, 1) = 0$. We also define $E_{2m}(0, y) := 0$ for every $y \in \mathbb{N}^*$.

We are now ready to state the following

Theorem 1 (Theorem 1 of [8]). *Let $x > 0$, $y \in \mathbb{N}$, $y \geq 3$, $m \in \mathbb{N}^*$. Then*

$$\begin{aligned} \log \Gamma\left(\frac{1}{x} + y\right) - \log \Gamma\left(\frac{1}{x}\right) &= -y \log x + 1 - y - \left(\frac{1}{x} + y - \frac{1}{2}\right) \log d(x, y) \\ &\quad + S_{2m}(x, y) + E_{2m}(x, y), \end{aligned}$$

where Γ is Euler's Gamma function, $d(x, y)$, $S_{2m}(x, y)$ and $E_{2m}(x, y)$ are respectively defined in (1)-(3). We further have that

$$|E_{2m}(x, y)| < 2 \left(1 + \frac{1}{4^{m+1}} \frac{2m+3}{2m+1} \right) \frac{x^{2m+1} (2m)!}{(2\pi)^{2m+2}} (1 - d(x, y)^{2m+1}).$$

Moreover, for $y \in \{1, 2\}$ we have

$$\log \Gamma\left(\frac{1}{x} + y\right) - \log \Gamma\left(\frac{1}{x}\right) = -y \log x - (y - 1) \log d(x, y).$$

We further specify that a way of guessing the order of magnitude of $\log \mathcal{L}$ for an overdispersed dataset is to evaluate the *asymptotic state of the system* defined as follows.

Definition 1 (Asymptotic state of the system). *We define the asymptotic state of the system as $\psi \rightarrow 0^+$ as*

$$\mathcal{A} := \lim_{\psi \rightarrow 0^+} \log \mathcal{L}(\mathbf{p}, \psi, \mathbf{x}) = \sum_{k=1}^K x_k \log p_k.$$

This definition is coherent with [18, eq. (20)], and with [8, Corollary 1]. It is clear that $\mathcal{A}$ only depends on the dataset probabilities $\mathbf{p}$ and data counts $\mathbf{x}$; so $\mathcal{A}$ depends only on some dataset characteristics. We also recall that the definition of overdispersed data corresponds on having a “small” value of ψ . Hence the value of the asymptotic state of the system $\mathcal{A}$ is particularly meaningful for overdispersed datasets.

For the computational cost estimate of the algorithm that uses Theorem 1 to evaluate $\log \mathcal{L}(\mathbf{p}, \psi, \mathbf{x})$, we refer to [8, Section 4].

3. The C Code and its Python Wrapped Version

For performance and accuracy reasons, in performing the powers of x and $d(x, y)$ needed into the sum $S_{2m}(x, y)$ defined in (2), a repeated products strategy is implemented to avoid the use of the `pow` function of the C99 language since, for $a > 0$ and $b \in \mathbb{R}$, it is defined as $\text{pow}(a^b) := \exp(b \log(a))$. In this way, $2m$ calls of the logarithm and $2m$ calls of the exponential functions are replaced with just $2m$ products, thus, obtaining a consistent performance gain. Moreover, the values required for the normalised Bernoulli numbers $\frac{B_{2n}}{2n(2n-1)}$ were precomputed and stored in an external file to be loaded in memory just once at the beginning of the whole procedure.⁵ The strategy to determine the correct value of m is explained in [8, pp. 1998–1999].

The same ideas were also used to evaluate the error term in Theorem 1. In particular, such an error term estimate is performed *before* starting to evaluate the main quantities; in this way, considering also the asymptotic state of the system $\mathcal{A}$, see Definition 1, we were able to *a priori* establish if using the `double` numerical data type is possible to ensure for that particular set of data a final accuracy of `prec` decimals, where `prec` is a parameter decided by the user.

In practice, if a user requires that the value of $\log \mathcal{L}(\mathbf{p}, \psi, \mathbf{x})$ should be determined with an accuracy of `prec` mantissa digits using the floating-point `double`-type of the C-programming language, we first evaluate $\mathcal{A}$ and then we check⁶ if `prec` + 2 + $\lceil \log_{10} |\mathcal{A}| \rceil \leq 15$, since the largest representable integer of the `double`-type has 15 decimal digits. If not, the user is asked to lower the value of `prec`, or to use an implementation that allows the use of larger numbers. In this procedure, a possible alternative is to replace $\mathcal{A}$ with $\mathcal{B} := \sum_{k=1}^K x_k \log f_k$, where $f_k := x_k/N$, $k = 1, \dots, K$, are the frequencies of each category. In this way, both the quantities $\mathcal{A}$ and $\mathcal{B}$ are useful in predetermining if the floating-point arithmetic will be able to handle the computation of $\log \mathcal{L}(\mathbf{p}, \psi, \mathbf{x})$ with the user-required accuracy. Remark that this result-reliability problem does not depend on the approach we are using to compute $\log \mathcal{L}$ but it is an intrinsic property of the dataset we are working with, and it depends on the numerical data-type used in the actual implementation.

Changing this kind of pre-evaluation procedure using a different numerical data type is trivial. Also, in this way, implementations using different numerical data types

⁵Since the Bernoulli numbers are rationals, it is also possible to store their normalized values as a ratio of two coprime integers thus removing any computational error that could have been originated from their *a priori* evaluation.

⁶with $\lceil u \rceil$ we denote the smallest integer greater than or equal to u .

(for example, a type that allows multiprecision computations) can be chosen at *runtime* according to the value of `ℳ` and `prec`.

The actual wrapping of C code into Python functions has been performed using the Python built-in module `ctypes`. This module allows loading a C language library containing C functions into the Python runtime environment. Then, leveraging the fact that in Python functions are first-class object, it is possible to define the types of the arguments and of the return value of each function in the library in terms of C language types, so that the binding between a statically typed language (namely, the C language) and a dynamically typed language (namely, the Python language) can be safely performed. Once these actions have been performed, the C functions can be invoked from Python at runtime as if they had been defined in Python. Obviously, the loading of the library and the definition of the types of arguments and return valued constitute an overhead. However, these actions need to be performed only once at the beginning of the procedure, hence, such an overhead can be considered negligible with respect to the total execution time.

4. The experiments

This section describes the experiments performed and illustrates and discusses their results. It also highlights the datasets used in conducting the experiments of this study and discusses their main characteristics.

4.1. Datasets

In order to contrast the performance of the `MP-math`, `MP-scipy`, and `MP-ALM` variants, 10 datasets from multiple domains were selected. As Table 1 shows, the Trump-Impeachment-Polls (TIP), the Trump-Clinton-Polls (TCP), and the IoT datasets are all count-based. While TIP and TCP are elections-census data, the IoT data are composed of counts of various protocols packets extracted from a smart-home IoT camera as described in [1]. The 20NewsGroup-auto (20NG-A), 20NewsGroup-mideast (20NG-M), and 20NewsGroup-space (20NG-S) datasets were extracted from the 20 Newsgroups collection widely used for topic analysis and text classification⁷. While the 20NG-A dataset comprises counts of the 10 top-most repeated words (i.e., keywords) in about 13K documents (i.e., lines) selected from the 20 Newsgroup on motor vehicles, the 20NG-M dataset comprises the counts of the 10 top-most keywords in almost 16K documents selected from the Middle East newsgroup. The 10 top-most keywords in 10K documents of the Space newsgroup comprise the 20NG-S dataset. A Python Keyword-Extractor script that utilized the open-source Natural Languages Toolkit (nltk) library was used to pre-process the documents in each dataset, and extract the designated keywords. It is worth remarking that the number of instances in the count matrix generated by the Keyword-Extractor rendered the execution time of the `MP` variants quite lengthy, thus, a second Python script was employed to reduce the number of instances by accumulating the counts of every 30 consecutive rows. The Cavallo

⁷https://scikit-learn.org/0.19/datasets/twenty_newsgroups.html

Table 1: Datasets

Dataset	Domain	Categories	Instances	ψ	Δ
Trump-Impeachment-polls (TIP) ⁸	Count: Elections Polls	3	408	0.020904538	$2.50 \cdot 10^{-9}$
Trump-Clinton-polls (TCP) ⁹	Count: Elections Polls	2	300	0.005596812	$3.33 \cdot 10^{-9}$
Cane ¹⁰	Image/Animals	120	85	0.000870921	$8.26 \cdot 10^{-5}$
Cavallo ¹¹	Image/Animals	120	130	0.000715444	$8.26 \cdot 10^{-5}$
IoT-Camera-a4(IoT) ¹²	Count: IoT Traffic	5	3420	0.072086934	$1.67 \cdot 10^{-9}$
20NewsGroup-Auto	Count: Document	11	346	0.000062484	$8.33 \cdot 10^{-5}$
20NewsGroup-Mideast	Count: Document	11	534	0.000202887	$8.33 \cdot 10^{-5}$
20NewsGroup-Space	Count: Document	10	335	0.000372259	$9.09 \cdot 10^{-5}$
US-County-Race-2020	Count: Census	3	3131	0.171987633	$2.50 \cdot 10^{-9}$
US-County-Ownership-2020	Count: Census	2	3139	0.025641677	$3.33 \cdot 10^{-9}$

and Cane datasets that comprise images of horses and dogs, respectively, were used to generate count matrices in the same manner illustrated in [1]. The US census data for the year 2020¹³, was utilized to extract two datasets: US-County-race-2020 (USC-R) and US-County-ownership-2020 (USC-O). The USC-R dataset comprises counts of three ethnicity groups of the populations of almost 3130 US counties, while the USC-O dataset consists of counts of Male Vs Female ownership of businesses in these counties¹⁴. Table 1 summarizes the datasets included in the experiments of this study, showing for each its domain, the number of its categories, the number of its instances, its estimated ψ value, and its tolerance, Δ .

4.2. Setup

A script that implements the three variants of Minka’s fixed-point iteration algorithm: MP-math, MP-scipy, and MP-ALM was developed using Python (3.11) with numpy (1.22.2), mpmath (1.2.1), and scipy (1.8.0) packages¹⁵. Each variant was applied to each dataset, yielding in each test, its own estimate for α while consuming its designated time and iterations. The paired differences of the F function in Algorithm 1, lines 7 and 8, were computed using the C-compiled `digamma(x)` function implementation provided in the scipy package. To contrast the performance of the three variants, the speedup between MP-math and MP-ALM, and the speedup between MP-scipy and MP-ALM were calculated. Also, the quality of the approximation was assessed using a collection of distance functions, and two goodness-of-fit tests. The experiments were executed on an

⁸<https://www.kaggle.com/datasets/teejmahal20/trump-impeachment-polls>

⁹<https://www.kaggle.com/datasets/etsc9287/2020-general-election-polls>

¹⁰<https://www.kaggle.com/datasets/alessiocorrado99/animals10>

¹¹<https://www.kaggle.com/datasets/alessiocorrado99/animals10>

¹²<https://data.mendeley.com/datasets/84cc8grtk/1>

¹³<https://www.kaggle.com/datasets/mexwell/us-country-demographics>

¹⁴The difference between the number of instances in the USC-R and USC-O dataset is due to the fact that some counties data on ethnicity was missing which implied excluding them from the dataset.

¹⁵We compared these versions with the ones of Python3.11.9-math and scipy v.1.14.0 without observing any relevant difference in their performances.

Intel Core i5 computer with 8 GB of RAM. The Euclidean distance metric, denoted by $\mathbf{D}$, was calculated using the `LA.norm` function from the numpy package, whereas the χ^2 -test was calculated using the `scipy.stats.chisquare` function.

4.3. Results and Discussion

The first set of experiments compared the number of iterations, IC , to converge for MP-math, MP-scipy, and MP-ALM, and its results are depicted in Fig. 1 (a)-(d). While Fig. 1(a) shows the IC counts for TIP and TCP datasets, the IC for Cane and Cavallo datasets are depicted in Fig. 1 (b). The IC s of the three 20-NewsGroups datasets are depicted in Fig. 1(c), whereas the USC-R, USC-O, and IoT datasets have their respective counts depicted in Fig. 1(d). As the figure shows, the IC counts seem to be quite similar across the different datasets. For the MP-math technique, the maximum IC , equal to 12.6K iterations was consumed for the TCP dataset, with 20% of the input. On the other hand, the smallest IC for the MP-math technique was 434 iterations consumed for the USC-R dataset with 25% of the input. Both MP-scipy and MP-ALM achieved their least IC values of 409 and 411, respectively, for the USC-R dataset with 25% of the data. Furthermore, MP-scipy required the most IC , almost 12.6K, for the 20NG-M dataset with 25% of the data, while the MP-ALM required the most IC , almost 12.6K, for the TCP dataset with 20% of the data. As the IC counts are rather comparable across the three techniques, an additional metric is needed to help distinguish their performance, namely, the time to converge, TC . As depicted in Fig. 2, MP-ALM was the fastest to converge for each of the 10 datasets, while MP-scipy was the slowest. For the TIP dataset with 20% of the input, the TC ranged between 0.0144 seconds for MP-ALM and 0.0541 seconds for MP-scipy, while MP-math TC was 0.02500 seconds. For the Cane dataset with 5% of the input, the TC ranged between 0.3974 seconds for MP-ALM and 4.5589 seconds for MP-scipy, while MP-math TC was 1.8835 seconds. For the 20NG-M dataset with 25% of the input, the TC ranged between 0.17707 seconds for MP-ALM and 0.9571 seconds for MP-scipy, while MP-math TC was 0.4319 seconds. For the USC-O dataset with 10% of the input, the TC ranged between 0.0321 seconds for MP-ALM and 0.0874 seconds for MP-scipy, while MP-math TC was 0.0412 seconds.

To further illustrate the superiority of MP-ALM with respect to MP-math and MP-scipy in terms of time, Fig. 3 shows the speedup MP-ALM achieved over MP-math and MP-scipy, for each of the 10 datasets, where Fig. 3(a) and (b) show the speedup of MP-ALM over MP-math, and Fig. 3(c) and (d) show the speedup of MP-ALM over MP-scipy. As Fig. 3 shows, MP-ALM was 5.02 times faster than MP-math for the Cavallo dataset with 25% of the input data, while it was 11.60 times faster than MP-scipy for the same dataset with the same input size. For the USC-O dataset, the speedup of MP-ALM over MP-math was almost 1.04 with 5% of the input size, and was 2.21 over MP-scipy for the same dataset but with 25% of the input size. According to Fig. 3, MP-ALM was 2.5 times faster than MP-math, while it was 5.5 faster than MP-scipy, on average.

Next, we shed light on another aspect of performance, namely the goodness-of-fit, which depicts the closeness of the estimated model to the real one. Table 2 shows the

Table 2: Error estimates in ψ

Dataset	tvd	KL-D	D	MSE
TIP	0.009212	0.000245	0.011537	0.000049
TCP	0.003501	0.000030	0.004951	0.000015
Cavallo	0.030959	0.003051	0.007505	0.0000005
Cane	0.046090	0.007388	0.0111654	0.000001
20NG-A	0.015905	0.000837	0.011794	0.000013
20NG-M	0.022211	0.001595	0.017313	0.000028
20NG-S	0.025714	0.002052	0.019218	0.000038
USC-R	0.138374	0.046025	0.174353	0.010137
USC-O	0.027907	0.001663	0.039467	0.000782
IoT	0.045508	0.012763	0.051873	0.000539

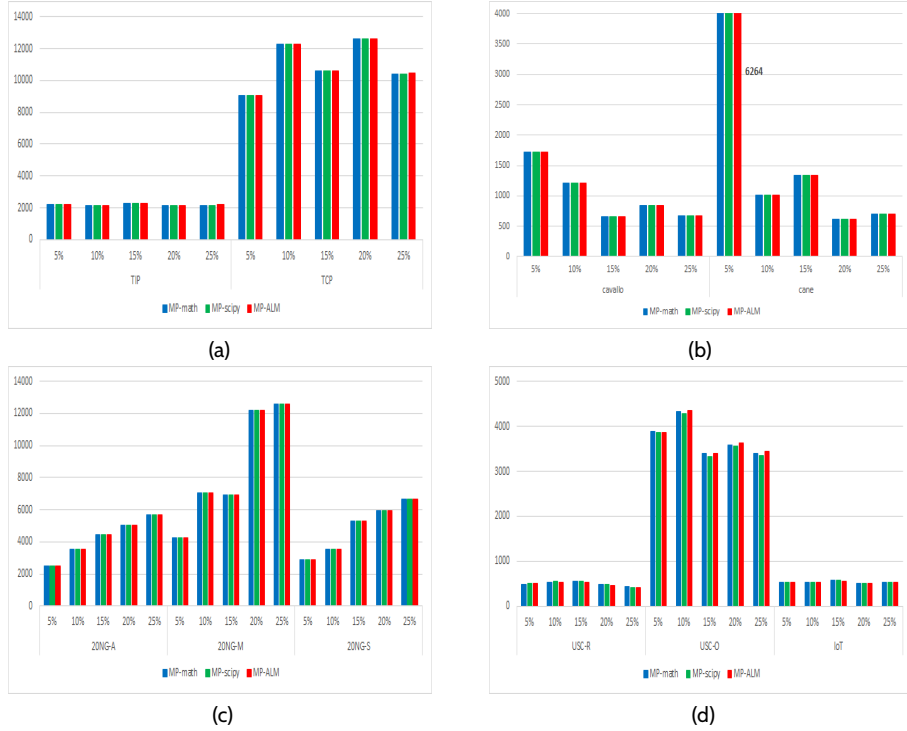


Figure 1: Iterations to Converge (IC) for MP-math, MP-scipy, and MP-ALM.

goodness-of-fit results of the MP-ALM technique with 25% of the input size¹⁶, using four different distance metrics: Total Variation Distance (**tvd**), the Kullback–Leibler

¹⁶As the accuracy of the three techniques is comparable, we opted to perform the goodness-of-fit experiments using the MP-ALM technique

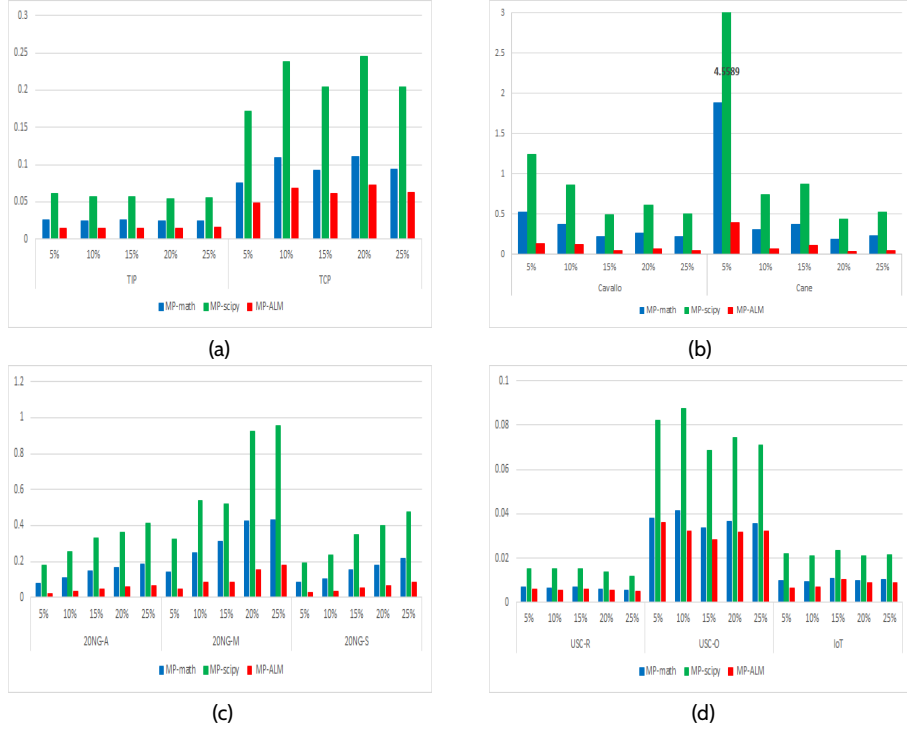


Figure 2: Time to Converge (TC) for MP-math, MP-scipy, and MP-ALM.

divergence Distance (**KL-D**), Euclidean Distance (**D**), and the Mean-Square-Error (**MSE**). The best distance values for every metric and across all datasets are typed in bold-face. As Table 2 shows, the **tvd** values ranged between 0.003501 and 0.138374, while the values of **KL-D** ranged between 0.000030 and 0.046025. The Euclidean Distance values ranged between 0.004951 and 0.174353, whereas the **MSE** values ranged between 0.0000005 and 0.010136. Moreover, Table 2 shows that most of the best goodness-of-fit results were achieved for the TCP dataset. Yet, all values listed in this table show that the **MP-ALM** technique managed to produce **DM** models that were quite close to the actual models. This is further supported by the goodness-of-fit tests depicted in Table 3 which show that the **MP-ALM** models for each of the 10 datasets managed to pass the Z -test, while six of them passed the χ^2 -test, and only two failed it. Moreover, the χ^2 -test was not applicable for the two image datasets; Cavallo and Cane.

5. Related Work

This Section sheds light on recent studies that address the utilization of the **DM** distribution and the techniques used to estimate its parameters. One technique commonly employed in literature to estimate **DM** model parameters is Expectation Maximization (EM). This widely-used technique consists of two steps: the E-step in which the expected

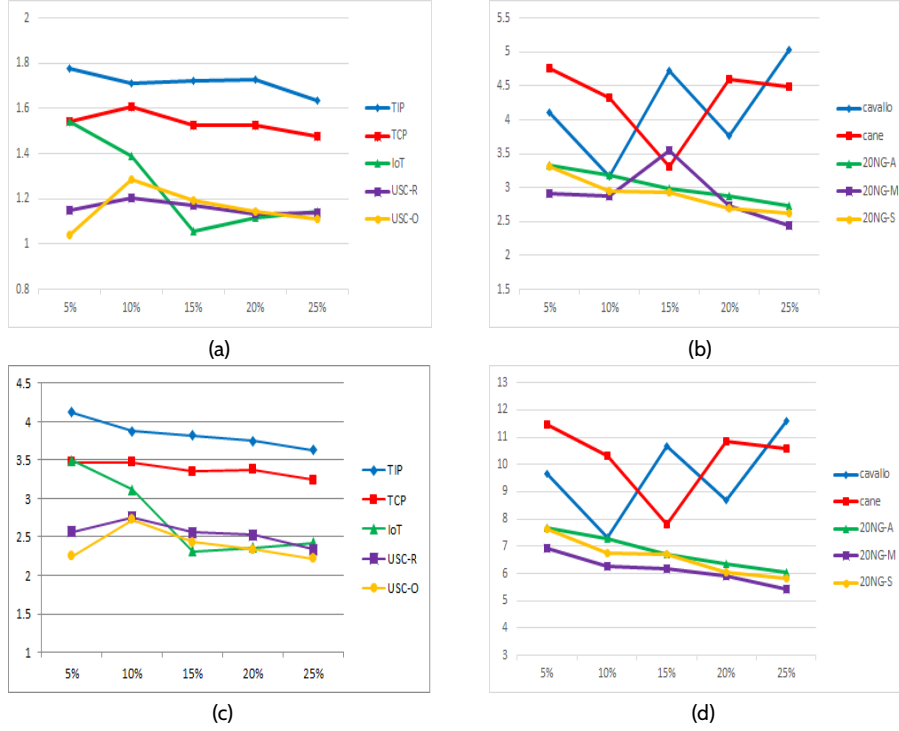


Figure 3: Speedup of MP-ALM with respect to MP-math (a and b), MP-scipy (c and d)

log-likelihood is computed given the observed data and the current parameter estimate, and an M-step in which the log-likelihood is maximized. These steps are repeated until convergence, yielding the parameters that achieved the maximum likelihood. The study in [5] depicts a set of weighted **DM** distributions to model overdispersed data, where the EM technique with a quasi-Newton optimization was employed to estimate the models parameters. To compare the performance of the proposed set of models with other approaches, 100 random datasets were synthesized with varying levels of overdispersion. The proposed set of models outperformed the other classical models in some, but not all, the metrics chosen for performance. The **DM** distribution, among others, was utilized in the context of document clustering in [17], where the EM technique was also utilized in order to estimate the model parameters. Experiments showed that the generalized Dirichlet and Beta-Liouville distributions outperformed the basic **DM** distribution in terms of relevant word projection and document clustering tasks. Expectation Maximization was also employed in the study depicted in [6] where a mixture of multinomial models was employed to cluster microbiome data. In the context of maximizing the log-likelihood of the **DM** model parameters, the EM technique suffers from major drawbacks including slow convergence, especially with high-dimensional data, and local minima. Thus, recent studies opted for alternative numerical techniques including but not limited to: Fixed-point Iteration, Newton-Raphson, and Gradient

Table 3: The statistical tests χ^2 -test and Z -test results

Dataset	χ^2 -test	Z -test
TIP	pass	pass
TCP	pass	pass
Cavallo	NA	pass
Cane	NA	pass
20NG-A	pass	pass
20NG-M	pass	pass
20NG-S	pass	pass
USC-R	fail	pass
USC-O	pass	pass
IoT	fail	pass

ascent, while others employed other approaches, like optimization and machine learning.

The study depicted in [20] opted for an optimization approach for clustering multi-variate count data using a Dirichlet-multinomial network with weighted fusion penalty. Since the **DM** log-likelihood function is nonconcave, exponential approximations were utilized to help solve the optimization problem. The effectiveness of the proposed approach was illustrated through simulation studies and two text clustering applications. A mixture of Stochastic Gradient Descent (SGD) and Neural Networks (NN) was used in [19] to implement the EM technique in the context of maximization of the likelihood of the **DM** distribution. Experimental results on four synthetic datasets and three real-world datasets showed the effectiveness of this approach against state-of-the-art. The reader is referred to [3], [9], [10] and [13] for further recent studies.

6. Conclusions and Future Work

This study evaluated a Python C-wrapped implementation of the **LM** technique introduced in [8] in the context of estimating the Dirichlet Multinomial (**DM**) distribution parameters for overdispersed count data. The Experiments conducted in this study utilized 10 publicly available datasets of different sizes, some of which were not overdispersed. Moreover, three techniques for estimating the *log-likelihood* ($\log \mathcal{L}$) function in the context of Minka’s procedure (MP) depicted in [11], namely **LM** (MP-ALM), Python’s math library’s (MP-math), and Python’s Scipy library’s techniques (MP-scipy), were compared in terms of their speed. Results showed the superiority of the MP-ALM technique compared to the other two for all 10 datasets, with a speedup of 10%, at least. An additional major advantage that MP-ALM has over the two other techniques is that it allows the user to select the desired degree of precision in computing $\log \mathcal{L}$, a feature not provided by the other techniques.

In this work we evaluated only the impact of the way Γ is computed, however, how the implementation of the digamma function that can be obtained using the **LM** approach, see [8, Theorem 2], could affect the performances of the whole MP-framework will be the subject of further studies. Moreover, given the results presented in this work, it is worthwhile to study the performance of Minka’s procedure using also **LM**’s C-wrapped

implementation of the digamma function compared to Python’s MP implementation and using several datasets having different levels of overdispersion.

References

- [1] S. Al-Haj Baddar, A. Languasco, M. Migliardi, “Efficient analysis of overdispersed data using an accurate computation of the Dirichlet multinomial distribution”, *TechArxiv*, *submitted*, 2023. DOI:<https://doi.org/10.36227/techrxiv.24086910.v1>.
- [2] S. Al-Haj Baddar, A. Languasco, M. Migliardi, “Modeling and forecasting overdispersed IoT data using an efficient and accurate computation of the Dirichlet Multinomial distribution”, *TechArxiv*, *submitted*, 2024. DOI:<https://doi.org/10.36227/techrxiv.172174633.33936008/v1>.
- [3] L. Anderlucci, C. Viroli, “Mixtures of Dirichlet-Multinomial distributions for supervised and unsupervised classification of short text data”, *Adv. Data Anal. Classif.*, vol. 14, pp. 759–770, 2020. DOI:<https://doi.org/10.1007/s11634-020-00399-3>.
- [4] C. De Campos, Q. Ji, “Bayesian networks and the imprecise Dirichlet model applied to recognition problems”, *European Conference on Symbolic and Quantitative Approaches to Reasoning and Uncertainty*, Belfast, UK, 2011, pp. 158–169. DOI:https://doi.org/10.1007/978-3-642-22152-1_14.
- [5] N. Corsini, C. Viroli, “Dealing with overdispersion in multivariate count data”, *Comput. Statist. Data Anal.*, vol. 170, article n. 107447, 2022. DOI:<https://doi.org/10.1016/j.csda.2022.107447>.
- [6] Y. Fang, S. Subedi, “Clustering microbiome data using mixtures of logistic normal multinomial models”, *Sci. Rep.*, vol. 13, article n. 14758, 2023. DOI:<https://doi.org/10.1038/s41598-023-41318-8>.
- [7] C. Lanczos, “A Precision Approximation of the Gamma Function”, *SIAM J. Numer. Anal.*, vol. 1, pp. 86–96, 1964, DOI:<https://doi.org/10.1137/0701008>.
- [8] A. Languasco, M. Migliardi, “On the fast computation of the Dirichlet-multinomial log-likelihood function”, *Comput. Stat.*, vol. 38, pp. 1995–2013, 2023. DOI:<https://doi.org/10.1007/s00180-022-01311-7>.
- [9] W. Li, J. Yin, H. Chen, “Supervised topic modeling using hierarchical Dirichlet process-based inverse regression: Experiments on e-commerce applications”, *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 6, pp.1192–1205, 2017. DOI:<https://doi.org/10.1109/TKDE.2017.2786727>.
- [10] Z. Li, H. Yan, C. Zhang, et al., “Individualized passenger travel pattern multi-clustering based on graph regularized tensor latent Dirichlet allocation”, *Data Min. Knowl. Disc.*, vol. 36, pp. 1247–1278, 2022. DOI:<https://doi.org/10.1007/s10618-022-00842-3>.

- [11] T.P. Minka, “Estimating a Dirichlet distribution”, 2000 (revised 2003, 2009, 2012). <https://tminka.github.io/papers/dirichlet/minka-dirichlet.pdf>.
- [12] B. Şahin, A. Evren, E. Tuna, Z. Şahinbaşoğlu, E. Ustaoglu, “Parameter Estimation of the Dirichlet Distribution Based on Entropy”, *Axiom*, vol. 12, article n. 10, 2023. DOI:<https://doi.org/10.3390/axioms12100947>.
- [13] D. Shen, C. Qin, C. Wang, Z. Dong, H. Zhu, H. Xiong, “Topic modeling revisited: A document graph-based neural network perspective”, *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 14681–14693, 2021. <https://openreview.net/pdf?id=yewqeLly5D8>.
- [14] E.J. Suh, “A Python package to estimate the Dirichlet distribution, calculate maximum likelihood, and test for independence from a variable based on fitting nested Dirichlet distribution hypotheses”, v. 0.9.0, release date: March 16th, 2020, <https://github.com/ericsuh/dirichlet>.
- [15] The Python Software Foundation, “Python Documentation: math module: lgamma”, release date: June 6th, 2024, <https://docs.python.org/3/library/math.html#math.lgamma>; <https://github.com/python/cpython/blob/main/Modules/mathmodule.c>.
- [16] The SciPy community, “SciPy v1.14.0 User’s Guide; special functions: gammaln”, release date: June 24th, 2024, <https://docs.scipy.org/doc/scipy/reference/generated/scipy.special.gammaln.html>. <https://github.com/scipy/scipy/blob/main/scipy/special/xsf/cephes/gamma.h>.
- [17] S.S. Yazdi, F. Najar, N. Bouguila, “Generalized Probabilistic Clustering Projection Models for Discrete Data”, *2023 International Symposium on Networks, Computers and Communications (ISNCC)*, Doha, Qatar, 2023, pp. 1–7. DOI:<https://doi.org/10.1109/ISNCC58260.2023.10323873>.
- [18] P. Yu, C.A. Shaw, “An efficient algorithm for accurate computation of the Dirichlet-multinomial log-likelihood function”, *Bioinformatics*, vol. 30, pp. 1547–1554, 2014. DOI:<https://doi.org/10.1093/bioinformatics/btu079>.
- [19] Y. Zhang, J. Yan, R. Zhenyu, J. Yin, “Learning mixture of neural temporal point processes for event sequence clustering”, *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-22)*, Vienna, Austria, 2022, pp.3766–3772 DOI:<https://doi.org/10.24963/ijcai.2022/523>
- [20] X. Zhao, J. Zhang, W. Lin, “Clustering multivariate count data via Dirichlet-multinomial network fusion”, *Comput. Statist. Data Anal.*, vol. 179, article n. 107634, 2023. DOI:<https://doi.org/10.1016/j.csda.2022.107634>.